

*** How to generate 8 characters random password by using Supplier<T>**

```
package com.ravi.supplier;

import java.util.Random;
import java.util.function.Supplier;

public class RandomPasswordSupplier
{
    public static void main(String[] args)
    {
        String upper = "ABCDEFGHJKLMNOPQRSTUVWXYZ"; //26
        String lower = "abcdefghijklmnopqrstuvwxyz"; //26
        String digit = "0123456789"; //10
        String special = "!@#%*&"; //8 //70 characters

        String totalCharacter = upper + lower + digit + special;

        Random random = new Random();

        Supplier<String> pwdGenerator = ()->
        {
            String password = "";

            for(int i=1; i<=8; i++)
            {
                int index = random.nextInt(totalCharacter.length()); //0 to 69
                password += totalCharacter.charAt(index);
            }
            return password;
        };

        String randomPassword = pwdGenerator.get();
        IO.println("Random password is :"+randomPassword);
    }
}
```

Can we develop a custom Functional interface with Type parameters :

Yes, We can develop our own **custom Functional interface** using Type parameters as shown below :

```
package com.ravi.custom_interface;

@FunctionalInterface
interface TriFunction<T,U,V,R>
{
    R myApply(T t, U u, V v);
}

public class CustomFunctionalInterface
{
    public static void main(String[] args)
    {
        TriFunction<Integer, Integer, Integer, String> fn1 =
            (t, u, v) -> ""+ t + u + v;

        IO.println("Result is :"+fn1.myApply(100, 200, 300));
    }
}
```

T : First input to the Function
U : Second input to the Function
V : Third input to the Function
R : Return Result

BiPredicate<T,U>

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents a predicate (a boolean-valued function) OF TWO ARGUMENTS.

The BiPredicate interface has method named test, which takes two parameters and returns a boolean value, basically this BiPredicate is same as the Predicate, instead, it takes 2 arguments for the metod test.

```
@FunctionalInterface
public interface BiPredicate<T, U>
{
    boolean test(T t, U u);
}

Type Parameters:
T - the type of the first argument to the predicate
U - the type of the second argument the predicate

Note : return type is boolean.

//Programs:
package com.ravi.bi_predicate;

import java.util.function.BiPredicate;

public class BiPredicateDemo1
{
    public static void main(String[] args)
    {
        //Verify that addition of two number is even or odd
        BiPredicate<Integer, Integer> b1 = (x,y) -> (x+y)%2==0;

        int x = Integer.parseInt(IO.readLine("Enter the value of x : "));
        int y = Integer.parseInt(IO.readLine("Enter the value of y : "));

        IO.println("Sum of two number is even ? "+b1.test(x, y));
    }
}

package com.ravi.bi_predicate;

import java.util.function.BiPredicate;

public class BiPredicateDemo2 {

    public static void main(String[] args)
    {
        //Validate the user with valid user name & password

        BiPredicate<String, String> b2 = (u, p) -> u.equals("Java") && p.equals("12345");

        String user = IO.readLine("Enter your username :");
        String pass = IO.readLine("Enter your password :");

        if(b2.test(user, pass))
        {
            IO.println("User Authenticated");
        }
        else
        {
            System.err.println("Sorry!!! Wronguser name OR password");
        }
    }
}
```

BiConsumer<T, U> functional interface :

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents an operation that accepts two input arguments and returns no result.

It takes a method named accept, which takes two parameters and performs an action without returning any result.

```
@FunctionalInterface
public interface BiConsumer<T, U>
{
    void accept(T t, U u);
}

//Program :
package com.ravi.bi_consumer;

import java.util.function.BiConsumer;

record Employee(int id, String name, double salary) {}

public class BiConsumerDemo1
{
    public static void main(String[] args)
    {
        //Salary increment by passing the bonus amount of an Employee
        BiConsumer<Employee, Double> b1 = (employee, bonus) ->
        {
            double incrementedSalary = employee.salary() + bonus;
            IO.println("Salary with increment is :"+incrementedSalary);
        };

        Employee e1 = new Employee(101, "Scott", 50000);
        IO.println(e1);

        b1.accept(e1, 50000);
    }
}

package com.ravi.bi_consumer;

import java.util.function.BiConsumer;

record Product(Integer id, String name, Double price) {}

public class BiConsumerDemo2
{
    public static void main(String[] args)
    {
        //add GST amount with given product price and print the total amount
        BiConsumer<Product, Double> finalPrice = (product, gst) ->
        {
            IO.println("Product datils :"+product);
            IO.println("Final price after adding GST :"+(product.price()+gst));
        };

        Product p1 = new Product(111, "Laptop", 95000D);
        double gstAmt = 2400;

        getFinalPrice(finalPrice, p1, gstAmt);

        //Method accepting Functional interface as a parameter
        public static void getFinalPrice(BiConsumer<Product, Double> bi, Product p, Double gst)
        {
            bi.accept(p, gst);
        }
    }
}
```

BiFunction<T, U, R> Functional interface :

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents a function that accepts two arguments and produces a result R.

The BiFunction interface has a method named apply that takes two arguments and returns a result of type R.

```
@FunctionalInterface
public interface BiFunction<T, U, R>
{
    R apply(T t, U u);
}

//Program
package com.ravi.bi_function;

import java.util.function.BiFunction;

record Employee(Integer id, String name, Double salary){}

public class BIFunctionDemo1
{
    public static void main(String[] args)
    {
        //Take Employee & Bonus as an input and return the total amount
        BiFunction<Employee, Double, Double> fn1 = (employee, bonus) ->
        {
            double updatedSalary = employee.salary() + bonus;
            return updatedSalary;
        };

        Employee emp = new Employee(101, "Smith", 75000D);
        double bonus = 10000;

        getTotalAmount(fn1, emp, bonus);

        public static void getTotalAmount(BiFunction<Employee, Double, Double> fn, Employee e, double amt)
        {
            Double updatedSalary = fn.apply(e, amt);
            IO.println("Employee Details :"+e);
            IO.println("Total Salary with bouns is :"+updatedSalary);
        }
    }
}
```

UnaryOperator<T> : [If input type & return type both are same use UnaryOperator]

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents an operation on a single operand that produces a result OF SAME TYPE AS ITS OPERAND. This is a specialization of Function for the case where the operand and result are of the SAME type.

It has a single type parameter, T, which represents both the operand type and the result type.

```
@FunctionalInterface
public interface UnaryOperator<T> extends Function<T,T>
{
    public abstract T apply(T X); //Input and Result both are same type
}

import java.util.function.*;
public class UnaryOperatorDemo
{
    public static void main(String[] args)
    {
        UnaryOperator<Integer> optr1 = num -> num*num;
        IO.println("Square of 5 is :"+optr1.apply(5));

        UnaryOperator<String> optr2 = str -> str.toUpperCase();
        IO.println("Java in upper case is :"+optr2.apply("java"));
    }
}
```